

SAKSHI: An Evidence-Gated AI Reasoning Agent for Death Investigation

Master Engineering Specification, Technical Architecture, and Implementation Blueprint

SECTION A — System Overview

SAKSHI ("The Incorruptible Witness") is an evidence-gated artificial intelligence reasoning and case-memory companion engineered specifically for statutory death investigations. In unnatural death and homicide investigations, Investigating Officers (IOs) face massive evidentiary friction: disorganized multi-modal case records, contradictory witness testimonies, complex post-mortem reports, voluminous Call Detail Records (CDRs), and high-pressure investigative timelines. Cognitive vulnerabilities such as premature cognitive closure, confirmation bias, and heuristic profiling have historically contributed to fatal investigative errors and miscarriages of justice.

Sakshi is architected not to automate justice, but to enforce rigorous evidentiary discipline. It ingests multi-modal case documents—starting from the First Information Report (FIR)—constructs an immutable, structured case memory, extracts invariant entities and hard machine-verifiable identifiers, builds a continuous chronological timeline, flags contradictions across statements, exposes procedural and evidentiary omissions (investigative gaps), and formulates evidence-backed investigative directions.

At the core of Sakshi is an **Asymmetric Two-Brain Architecture**:

- **The Intuition Brain (LLM-based Discovery Layer):** Operates within an isolated informational sandbox to synthesize context, identify patterns, spot temporal inconsistencies, and formulate candidate investigative hypotheses. It is structurally prohibited from directly communicating with or presenting unverified claims to the investigator.
- **The Discipline Brain (Deterministic Verification & Gating Layer):** Acts as a mandatory judicial firewall. It intercepts every candidate hypothesis generated by the Intuition Brain, decomposes it into atomic factual propositions, and executes deterministic verification queries against cryptographically hashed primary source records. If a claim lacks unbroken documentary proof, it is either flagged as an unverified lead or rejected.

Crucially, Sakshi features a **Hard-Identifier Cross-Case Safeguard**: cross-case associations are strictly permitted only on exact matches of machine-verifiable tokens (Phone Numbers, IMEI, Vehicle Plates, Bank Accounts). All heuristic cross-case linkage based on appearance, behavioral patterns, personality profiling, or Modus Operandi (MO) is **hard-blocked at the kernel level**.

SECTION B — Design Principles & Anti-Goals

1. Core Operating Principles

1. **The Cardinal Anti-Suspect Principle:** Sakshi must **NEVER** name, rank, recommend, implicate, predict, or calculate the probability of guilt for any suspect. Culpability scoring and automated accusations are strictly prohibited.
2. **Absolute Source Traceability (Grounding Ratio = 1.0):** Every fact presented as verified on the investigator's console must possess a direct, cryptographically validated citation pointing to an exact document ID, page, paragraph, and line span.
3. **The Mandatory Two-Brain Gate:** Generative language models may brainstorm and detect candidate anomalies, but only deterministic verification engines may approve output for the user.
4. **Human Primacy (Human-in-the-Loop):** The human Investigating Officer remains the sole decision-maker, exercising statutory authority, conducting field inquiries, and drawing judicial conclusions. Sakshi assists by organizing evidence, exposing gaps, and mapping contradictions.
5. **Deterministic Legal Procedural Control:** Statutory legal requirements (e.g., inquest formalities under CrPC/BNSS, medical post-mortem requisitions, forensic sample custody) are evaluated by a rule engine, completely decoupled from LLM inference.
6. **Immutable Evidentiary Provenance:** All raw files, AI prompt transactions, verification steps, and officer decisions are committed to an append-only, tamper-evident cryptographic Merkle audit log.

2. Explicit System Anti-Goals (Kernel-Level Prohibitions)

- **✗ NO Suspect Ranking:** The system will not output "Top Suspects" or probabilistic suspect likelihoods.
- **✗ NO Guilt Prediction:** Algorithms will not assign guilt percentages to any individual.
- **✗ NO Behavioral or Psychological Profiling:** The system will not infer deceptive personality traits, criminal inclinations, or moral character.
- **✗ NO Appearance-Based Cross-Case Matching:** Computer vision facial similarity or witness description matching across cases is barred.
- **✗ NO Modus Operandi (MO) Linkage:** Cases will not be correlated based on subjective behavioral or methodological similarities.
- **✗ NO Silent Hallucination:** The system will never present an uncorroborated AI hypothesis disguised as an established fact.

SECTION C — Layered System Architecture

Sakshi is organized into six primary horizontal tiers and five cross-cutting governance layers:





SECTION D — Component Architecture & Responsibilities

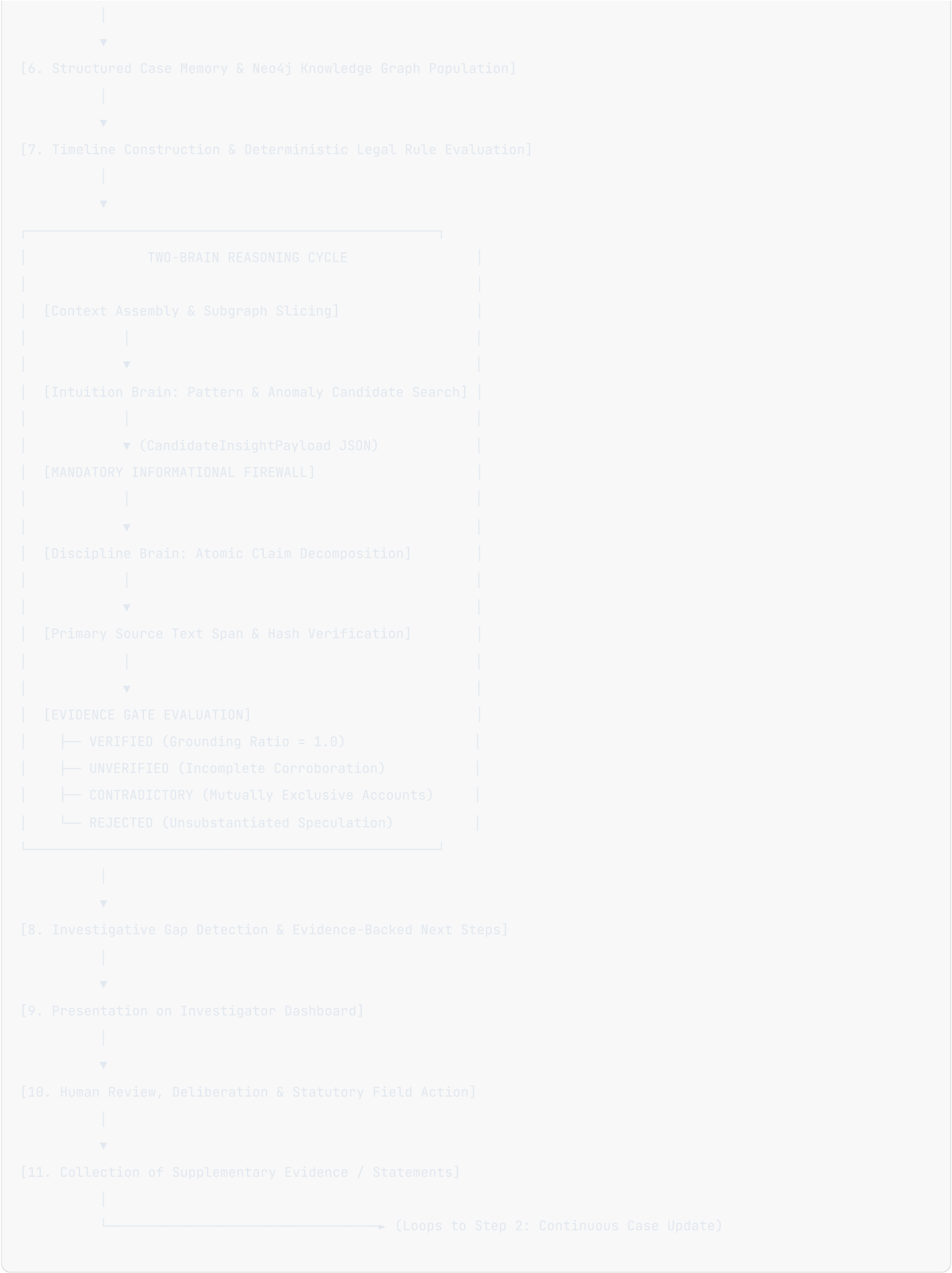
- Investigator Web Console (Next.js/React):** The primary user interface rendering the interactive chronological timeline, side-by-side evidence citation viewer, contradiction matrix, gap board, and hypothesis board.
- Voice Assistant Interface (WebRTC / Local Whisper):** Allows field investigators to dictate notes, record Malayalam-English mixed witness interviews, and execute voice-driven queries against case memory.
- API Gateway & Access Control (Kong / Keycloak):** Enforces TLS 1.3, mTLS, OAuth2/OIDC authentication, role-based and attribute-based access control (RBAC/ABAC), and jurisdictional tenant isolation.
- Case Management Service (FastAPI):** Oversees case registration, FIR intake, IO assignment, investigation milestones, and case diary maintenance.
- Evidence Ingestion & Hashing Service:** Computes SHA-256 and SHA-3-512 dual-checksums on inbound binaries, validates file signatures, runs anti-virus inspection (ClamAV), and writes immutably to WORM storage.
- Multi-Modal OCR & Document Layout Service:** Employs Tesseract, PaddleOCR, and layout parsers to convert scanned police panchnamas, typed statements, and tabular records into structured text and tables.
- Bilingual Speech-to-Text Service:** Specialized ASR pipeline running fine-tuned Whisper models capable of transcribing code-switched Malayalam and English speech.
- NLP & Information Extraction Engine:** Uses fine-tuned Legal-Clinical transformer models to extract named entities, atomic event tuples, temporal intervals, and machine-verifiable identifiers.
- Structured Case Memory Service:** Orchestrates data synchronization between the relational database (PostgreSQL 16) and the property knowledge graph (Neo4j Enterprise).

10. **Chronological Timeline Engine:** Maps events along continuous temporal coordinates, applying Allen's Interval Algebra to detect impossible spatio-temporal travel speeds and chronological overlaps.
11. **Deterministic Legal / Procedural Rule Engine:** A non-LLM, rule-based statutory engine verifying compliance with criminal procedure codes (e.g., 24-hour magistrate reporting, inquest formalities, mandatory forensic sample dispatches).
12. **Intuition Brain Inference Service:** Self-hosted vLLM engine running open-source models (e.g., Llama-3-70B-Instruct) in an air-gapped environment. Synthesizes context to generate candidate insights.
13. **Discipline Brain Verification Kernel:** Decomposes candidate insights into atomic claims, executes semantic span and exact string matches against primary document OCR text, and measures grounding ratios.
14. **Evidence Gate:** Strictly filters claims: assigns **VERIFIED** status only to claims with 100% evidentiary backing; routes unsupported claims to **UNVERIFIED LEADS** or **INVESTIGATIVE GAPS**.
15. **Investigative Gap Engine:** Analyzes the knowledge graph to detect missing corroborative evidence, unverified statements, and incomplete statutory procedures.
16. **Cross-Case Hard-Identifier Linker:** Exact-match query engine scanning indexed phone numbers, IMEIs, vehicle plates, and bank accounts across independent case databases.
17. **Append-Only Merkle Audit Service:** Maintains a cryptographic, tamper-evident audit trail of every ingested byte, model interaction, verification check, and IO decision.
18. **Statutory Certificate Generator:** Produces digitally signed certificates of electronic record integrity compliant with Section 65B of the Indian Evidence Act / Section 63 of the Bharatiya Sakshya Adhiniyam (BSA).
19. **Forensic Science Laboratory (FSL) Integration Gateway:** Secure REST/mTLS gateway for automated, cryptographically signed intake of official laboratory toxicology, ballistics, and DNA reports.
20. **Case Diary & Report Generation Engine:** Assembles statutory daily case diaries, evidence summaries, and gap audit reports, cleanly segregating verified facts from unverified leads.

SECTION E — End-to-End Workflow

The Sakshi system operates as a **closed-loop feedback cycle**:





SECTION F — Data Flow Architecture (DFD Levels 0, 1, and 2)

Level-0 DFD (System Context)

Represents the system boundary, external human and statutory entities, and primary input/output vectors.

Level-1 DFD (Subsystem Deconstruction)

Deconstructs Sakshi into 8 core functional processes: 1.0 Case Registration & Management 2.0 Evidence Ingestion & Cryptographic Hashing 3.0 Evidence Processing & Feature Extraction 4.0 Case Memory & Graph Maintenance 5.0 Legal Rule Compliance Check 6.0 Two-Brain Reasoning & Evidence Gating 7.0 Cross-Case Linkage Engine 8.0 Investigator Presentation & Audit Logging

Level-2 DFD: Evidence Processing Pipeline (Process 3.0)

Explodes Process 3.0 into discrete sub-components: 3.1 Media Type Detection & Routing 3.2 Image Preprocessing & Multi-Engine OCR 3.3 Malayalam-English ASR Speech Transcription 3.4 Text Cleaning & Normalization 3.5 Legal Named Entity Recognition 3.6 Temporal Expression Normalization (ISO 8601 Interval Solver) 3.7 Hard-Identifier Regex & Algorithmic Validation 3.8 Atomic Event Tuple Formulation (S, P, O, L, T, SourceID)

Level-2 DFD: Two-Brain Reasoning & Evidence Gate (Process 6.0)

Explodes Process 6.0 into: 6.1 Subgraph Retrieval & Prompt Context Assembly 6.2 Intuition Brain Candidate Synthesis 6.3 Firewall Sanitization & Candidate Payload Validation 6.4 Atomic Claim Extraction & Proposition Mapping 6.5 Evidence Store Query & Hash Validation 6.6 OCR Text Span Matching & NLI Entailment Scoring 6.7 Forensic Contradiction Cross-Checking 6.8 Evidence Gatekeeper Verdict Formulation & Routing

SECTION G — Two-Brain Reasoning Architecture

The Two-Brain Architecture resolves the tension between LLM generative fluency and the absolute evidentiary precision required in judicial inquiries.

1. The Intuition Brain

- **Core Technology:** Self-hosted open-source foundation model (e.g., Llama-3-70B-Instruct, Mistral-Large) running on an air-gapped on-premise GPU cluster via vLLM.
- **Role:** Synthesis, pattern recognition, anomaly spotting, hypothesis generation, and gap detection.
- **Operational Boundary:** Consumes structured timeline JSON and case subgraph packets. It is strictly constrained by system prompts to search for **inconsistencies, missing testimonies, uncollected physical items, and timeline conflicts**.
- **Containment:** The Intuition Brain has no direct access to user-facing API routes or presentation layers. It emits a strongly typed `CandidateInsightPayload` into an internal buffer.

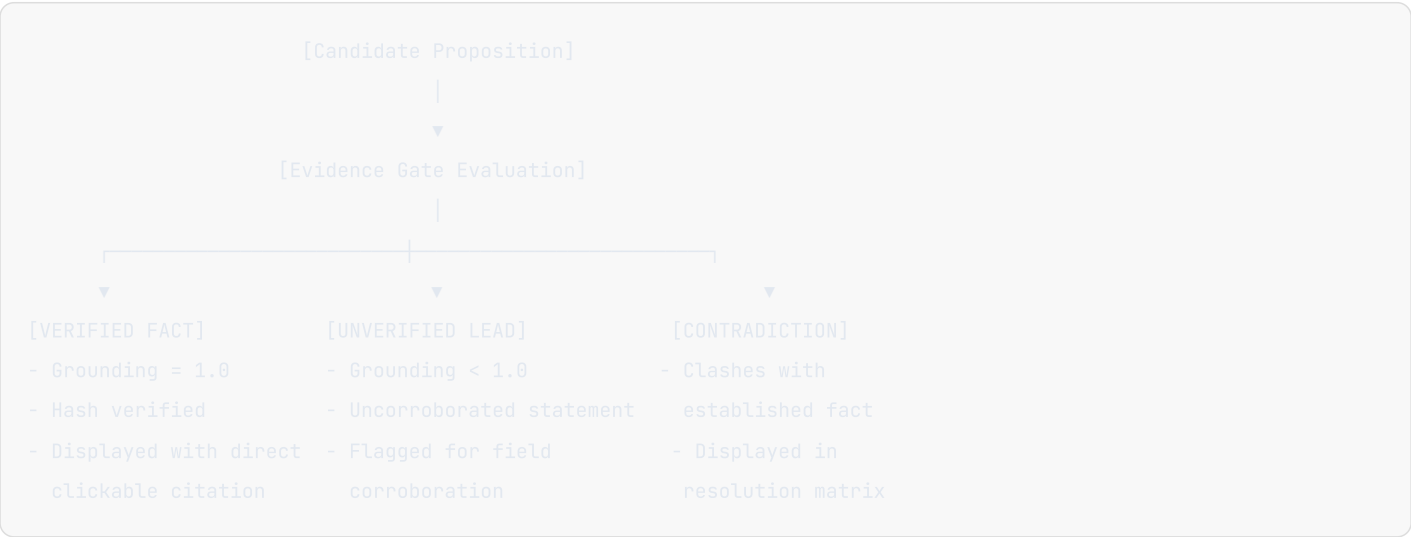
2. The Discipline Brain

- **Core Technology:** Deterministic Python/Rust verification engine utilizing exact string matching, RapidFuzz, SQL relational queries, Cypher graph lookups, and DeBERTa-v3-large-MNLI for natural language entailment verification.
- **Role:** Verification, source traceability, claim validation, contradiction analysis, and court-admissibility enforcement.
- **Operational Pipeline:**
 1. Receives `CandidateInsightPayload`.
 2. Decomposes compound assertions into discrete atomic claims.
 3. Validates that every cited `source_evidence_id` exists in the WORM storage vault with an intact hash.
 4. Scans the OCR text stream of the primary document to locate the exact text span cited.
 5. Computes the **Grounding Ratio**:
$$\text{Grounding Ratio} = \frac{\text{Count of Cryptographically Grounded Atomic Claims}}{\text{Total Atomic Claims in Hypothesis}}$$
 6. Evaluates whether the claim contradicts established forensic ground truths (e.g., Post-Mortem Time Since Death interval).

SECTION H — The Evidence Gate

The Evidence Gate is the central control mechanism standing between AI reasoning and investigator-facing displays. It evaluates every factual assertion against an eight-point verification rubric:

1. **Claim Identity:** Is the assertion an atomic, non-compound proposition?
2. **Documentary Existence:** Does the supporting evidence record exist in the repository?
3. **Integrity Validation:** Does the file's current SHA-256 hash match its registration hash?
4. **Source Authenticity:** Is the author/recording officer of the document identified?
5. **Exact Span Localization:** Can the claimed fact be pointed to on a specific page and line?
6. **Semantic Entailment:** Does the text span logically support the claim without extrapolation?
7. **Forensic Consistency:** Does the assertion conflict with objective medical/forensic facts?
8. **Sufficiency Threshold:** Is the claim corroborated by primary or secondary documentary evidence?



SECTION I — Structured Case Memory & Knowledge Graph

The Case Memory is a hybrid multi-model architecture. Vector embeddings and semantic indexes are utilized **exclusively for retrieval acceleration**; they are strictly barred from serving as the ground truth. The authoritative single source of truth resides in the **Relational Database** and the **Property Knowledge Graph**.

Core Knowledge Graph Ontology (Neo4j)

- **Node Labels:** `Person`, `EvidenceItem`, `Document`, `Statement`, `Event`, `Location`, `TimeInterval`, `MedicalFinding`, `HardIdentifier`, `Contradiction`, `InvestigativeGap`, `NextStep`.
- **Edge Types:**
 - `(:Person)-[:MADE_STATEMENT]→(:Statement)`
 - `(:Statement)-[:RECOUNTS]→(:Event)`
 - `(:Event)-[:OCCURRED_AT]→(:Location)`
 - `(:Event)-[:WITHIN_INTERVAL]→(:TimeInterval)`
 - `(:Person)-[:ASSOCIATED_WITH]→(:HardIdentifier)`
 - `(:EvidenceItem)-[:PROVES]→(:Event)`
 - `(:EvidenceItem)-[:CONTAINS]→(:MedicalFinding)`
 - `(:Statement)-[:CONFLICTS_WITH]→(:Statement)`
 - `(:MedicalFinding)-[:REFUTES]→(:Statement)`
 - `(:InvestigativeGap)-[:PERTAINS_TO]→(:Event)`

Every relationship carries provenance metadata: `evidence_id`, `page_number`, `confidence_score`, `verified_by_officer_id`, and `last_updated_timestamp`.

SECTION J — Timeline Construction & Contradiction Engine

1. Timeline Engine

- Aggregates all extracted temporal markers across FIR, witness statements, post-mortem findings, CCTV logs, and CDR call sessions.
- Converts fuzzy or relative temporal expressions (e.g., "half an hour after sunset", "around tea time") into formal ISO 8601 temporal intervals: `[start_timestamp, end_timestamp]` with explicit uncertainty bounds.
- Solves interval sequences using Allen's Interval Temporal Logic (`BEFORE`, `MEETS`, `OVERLAPS`, `DURING`, `EQUALS`).
- Evaluates spatial transit feasibility: computes minimum physical travel time between consecutive locations using geographic coordinates and road networks. If required transit velocity exceeds physical feasibility (e.g., > 120 km/h in urban traffic), an anomaly is automatically flagged.

2. Contradiction Detection Engine

- Detects factual collisions across three primary axes:
 1. **Temporal Clashes:** Witness A states X was at Location L1 at 21:00; Tower CDR registers phone at Location L2 (25 km away) at 21:05.

2. **Forensic vs. Testimonial Clashes:** Witness B asserts the deceased was assaulted with a wooden stick; Post-Mortem Report determines cause of death as organophosphate chemical poisoning with zero blunt force trauma.
3. **Direct Testimonial Clashes:** Witness C claims the victim arrived alone; Witness D claims the victim arrived with two unknown companions.
- **Rule of Non-Resolution:** Sakshi **never** automatically decides which witness is lying or which account is true. It exposes both conflicting nodes side-by-side with complete provenance, leaving evaluation to the human investigator.

SECTION K — Investigative Gap Engine

The Investigative Gap Engine scans the case memory to identify evidentiary voids, missing procedural steps, and unverified assertions.

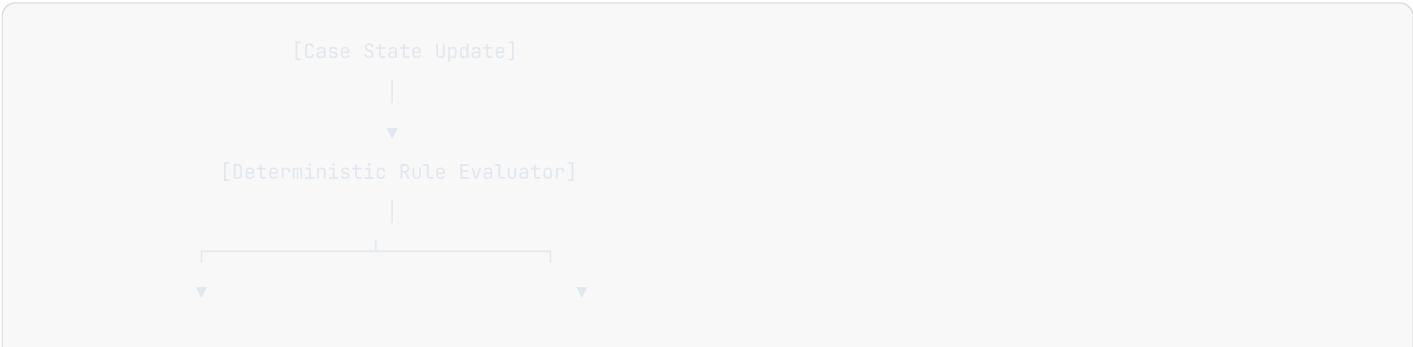
Taxonomy of Detected Gaps

1. **Uncorroborated Alibis / Movements:** A person claims an alibi during the estimated Time Since Death interval, but no physical, digital, or secondary witness corroboration exists.
2. **Missing Forensic Protocols:** An inquest report notes blood pooling at the scene, but no forensic blood-typing or DNA swabbing receipt is present in the case vault.
3. **Chain of Custody Lapses:** Physical evidence was seized on Date D1, but FSL dispatch entry is not recorded within statutory time limits.
4. **Unaccounted Chronological Blocks:** An unexplained gap of > 60 minutes exists in the deceased's verified timeline immediately preceding the estimated time of death.
5. **Telecom Discrepancies:** Phone calls are recorded between key entities during critical intervals, but corresponding subscriber detail records (SDR) or cell tower locations have not been summoned under statutory notices.

For every detected gap, the engine generates an **Evidence-Backed Next Step** (e.g., *"Issue notice under Sec 91 CrPC / BNSS to Telecom Provider for Tower CDR of Cell ID 404-21-982 for interval 22:00-23:30"*).

SECTION L — Deterministic Legal / Procedural Rule Engine

The Legal Rule Engine is a **strictly deterministic, non-LLM subsystem**. It codifies statutory procedural requirements under the Code of Criminal Procedure (CrPC) / Bharatiya Nagarik Suraksha Sanhita (BNSS), Indian Penal Code (IPC) / Bharatiya Nyaya Sanhita (BNS), and Indian Evidence Act (IEA) / Bharatiya Sakshya Adhinyam (BSA).





Key Rule Specifications

- **Rule REG-01 (Inquest Completion):** For unnatural death cases, verification that Form 174 / Inquest Panchnama is signed by required panchas and submitted to the Executive Magistrate.
- **Rule REG-02 (Post-Mortem Requisition):** Verification that medical autopsy report is officially linked to the registered FIR/Inquest number.
- **Rule REG-03 (Electronic Evidence Certification):** Enforces mandatory creation of Section 65B IEA / Section 63 BSA hash certificates for all digital devices, CDR dumps, and CCTV drives at the moment of seizure.
- **Rule REG-04 (Viscera Preservation):** If post-mortem cause of death is pending chemical analysis, rules check for the entry of viscera dispatch to the State Forensic Science Laboratory within 72 hours.

SECTION M — Cross-Case Hard-Identifier Linkage Engine

To prevent wrongful arrests and biased profiling, cross-case linkage is guarded by an algorithmic firewall:





The system outputs "IDENTIFIER MATCH FOUND", never "Suspect Match".

SECTION N — Human-in-the-Loop (HITL) Architecture

Sakshi enforces clear separation between computational processing and judicial decision-making:

- **What the System Does:** Organizes multi-modal evidence; parses complex timelines; detects statement contradictions; exposes missing evidence/procedural gaps; verifies claims against source documents; generates evidence-backed next steps.
- **What the System NEVER Does:** Decides whether a person is lying; predicts culpability; suggests who to arrest; draws final legal conclusions.
- **Investigator Prerogatives:** The IO inspects primary document citations via click-to-preview; resolves contradiction flags by recording re-interrogations; accepts or rejects suggested next steps; enters mandatory statutory justifications for deferred steps.

SECTION O — Security, Cryptographic Integrity & On-Premise Architecture

1. Sovereign On-Premise Air-Gapped Deployment

To protect sensitive police data and comply with national data sovereignty laws, Sakshi is engineered to run entirely within controlled police network infrastructure. All LLM inferences, vector embeddings, and speech models execute on local, on-premise GPU servers with **zero external cloud connectivity**.

2. Multi-Layer Cryptographic Security

- **Dual Hashing at Ingestion:** Every uploaded file is processed through streaming SHA-256 and SHA-3-512 hashing before reaching storage.

- **WORM Storage:** Raw evidence files are locked in Write-Once-Read-Many (WORM) storage using S3 Compliance Object Locking. Deletions and modifications are cryptographically impossible.
- **Envelope Encryption:** Data at rest is encrypted using AES-256-GCM. A dedicated Hardware Security Module (HSM) / Key Management Service (KMS) provisions unique cryptographic keys per case workspace.
- **Transit Encryption:** All internal microservice communications enforce TLS 1.3 with mutual authentication (mTLS).
- **Append-Only Merkle Audit Trail:** System events, AI model versions, prompts, and officer interactions are committed to a Merkle tree-based immutable ledger (Immudb).

SECTION P — Technology Stack & Selection Rationale

Architectural Tier	Technology Selected	Technical Rationale & Suitability
Frontend UI	Next.js 14 / React / Tailwind CSS / D3.js	Server-side rendering for security; modular component architecture; high-performance D3.js rendering for interactive timelines and knowledge graph visualizations.
Voice Interface	WebRTC / Open-Source Faster-Whisper	Low-latency audio streaming; local on-premise execution; fine-tuned for code-mixed Malayalam-English speech transcription.
API Gateway	Kong Gateway / Traefik	High throughput; native support for mTLS, rate limiting, and OpenID Connect (OIDC) integration with Keycloak.
Backend Core	Python FastAPI & Asyncio	Native asynchronous concurrency; seamless integration with machine learning runtimes (PyTorch, vLLM, HuggingFace); auto-generated OpenAPI schemas.
Relational Case DB	PostgreSQL 16 (JSONB & pgvector)	Strict ACID compliance; rich support for semi-structured legal schemas via JSONB; integrated vector similarity search for context retrieval.
Knowledge Graph	Neo4j Enterprise Edition	Industry-standard Property Graph database; Cypher query language optimized for multi-hop entity traversal and relationship tracing.
Vector Search	Qdrant / pgvector	Isolated vector embedding indexing; supports payload filtering by case ID, preventing multi-tenant vector leakage.
Raw Vault / WORM	MinIO Enterprise (Object Lock)	S3-compatible, high-performance object storage with certified WORM compliance mode for immutable evidentiary preservation.
Audit Log Store	Immudb (Cryptographic Ledger)	High-speed, immutable, cryptographically verifiable append-only database using Merkle trees; mathematical proof of tamper resistance.
LLM Inference	vLLM / Llama-3-70B-Instruct	State-of-the-art open-source LLM; hosted on local on-premise GPU clusters (NVIDIA A100/H100); high throughput via PagedAttention.
OCR Pipeline	Tesseract 5 + PaddleOCR + LayoutLM	Tesseract for standard English; PaddleOCR for regional Indian scripts (Malayalam); LayoutLM for complex police tabular forms.

Architectural Tier	Technology Selected	Technical Rationale & Suitability
NLP & IE Engine	spaCy + Legal-BERT + Duckling	Fast, token-level legal entity recognition; Duckling for deterministic extraction of temporal expressions and currencies.
Auth & Identity	Keycloak (OAuth2.0 / OIDC / FIDO2)	Enterprise IAM supporting smart-card authentication, hardware FIDO2 tokens, and fine-grained Attribute-Based Access Control (ABAC).

SECTION Q — The 22 Development Phases

Phase 1 — Requirement Analysis & Statutory Boundary Definition

- **Objective:** Codify statutory death investigation workflows (CrPC/BNSS, IEA/BSA) and lock the Cardinal Anti-Suspect Principle.
- **Inputs:** State Police Manuals, Statutory Criminal Codes, Forensic Inquest SOPs.
- **Components:** System Policy Engine, Legal Requirements Specification.
- **Processing:** Translating statutory inquest steps into deterministic operational schemas; codifying hard-identifier constraints.
- **Storage:** Git repository, Policy JSON Schemas.
- **AI Models:** None.
- **APIs:** None.
- **Outputs:** Architectural Requirements Document, Anti-Goals Matrix, JSON Schema Specifications.
- **Validation:** Review and sign-off by legal and digital forensics experts.
- **Security:** Policy-level definitions of zero-trust RBAC/ABAC boundaries.
- **Dependencies:** None.

Phase 2 — System & Database Design

- **Objective:** Architect schemas across all 10 storage engines ensuring referential integrity and strict case isolation.
- **Inputs:** Requirements from Phase 1.
- **Components:** Relational DB Schema Designer, Neo4j Graph Modeler, WORM Vault Configurator.
- **Processing:** Normalization of legal entity models; design of append-only audit ledger structures.
- **Storage:** PostgreSQL 16 DDL, Neo4j Graph Cypher DDL, MinIO Bucket Policies.
- **AI Models:** None.
- **APIs:** Internal DB migration scripts (Alembic).
- **Outputs:** ER diagrams, Graph Ontologies, Storage Configurations.
- **Validation:** Automated migration tests, foreign key integrity benchmarks.
- **Security:** Multi-tenant cryptographic schema isolation (unique schema per case).
- **Dependencies:** Phase 1.

Phase 3 — Secure Case Management Layer

- **Objective:** Implement case lifecycle management, FIR registration, and investigator assignment.
- **Inputs:** Case registration metadata, FIR text.
- **Components:** Case Management Service, Keycloak Identity Service.
- **Processing:** Case creation, officer assignment, stage transitions (FIR → Inquest → Active Inquiry → Final Report).
- **Storage:** PostgreSQL `cases`, `officers`, `case_stages` tables.
- **AI Models:** None.
- **APIs:** `/api/v1/cases`, `/api/v1/cases/{id}/assign`.
- **Outputs:** Active Case Workspaces, Initial Case Context Records.
- **Validation:** 100% unit test coverage of permission transitions and stage locks.
- **Security:** RBAC enforcement; access restricted strictly to assigned IOs and verified supervisors.
- **Dependencies:** Phase 2.

Phase 4 — Multi-Modal Evidence Ingestion Pipeline

- **Objective:** Construct high-throughput, secure multi-modal evidence ingestion with dual-hashing.
- **Inputs:** Scanned PDFs, images, CDR spreadsheets, audio recordings.
- **Components:** Ingestion Controller, Anti-virus Scanner (ClamAV), Dual-Hasher.
- **Processing:** File validation, ClamAV scanning, streaming SHA-256 and SHA-3-512 calculation, WORM commit.
- **Storage:** MinIO WORM S3 Vault, PostgreSQL `evidence_metadata`.
- **AI Models:** None.
- **APIs:** `/api/v1/evidence/upload`, `/api/v1/evidence/verify-hash`.
- **Outputs:** Immutable Storage URIs, Cryptographic Checksum Tokens.
- **Validation:** Bit-for-bit checksum verification upon upload completion.
- **Security:** WORM compliance mode; reject infected or corrupted binaries.
- **Dependencies:** Phase 3.

Phase 5 — OCR & Bilingual Speech Processing

- **Objective:** Convert scanned records and voice recordings into clean, normalized text.
- **Inputs:** PDF scans, panchnama photos, voice notes, audio interviews.
- **Components:** Tesseract/PaddleOCR Engine, Whisper Malayalam-English ASR.
- **Processing:** Image deskewing, binarization, layout-aware OCR, bilingual ASR transcription.
- **Storage:** PostgreSQL `processed_documents`, MinIO audio transcripts.
- **AI Models:** PaddleOCR (Malayalam/English), Faster-Whisper (fine-tuned).
- **APIs:** `/api/v1/process/ocr`, `/api/v1/process/transcribe`.
- **Outputs:** Text streams with character bounding box coordinates and word-level timestamps.
- **Validation:** Character Error Rate (CER) < 3% on printed legal text; Word Error Rate (WER) < 7% on audio.
- **Security:** On-premise execution; audio buffers sanitized from memory immediately after inference.
- **Dependencies:** Phase 4.

Phase 6 — NLP & Information Extraction

- **Objective:** Extract entities, atomic events, temporal intervals, and hard identifiers.
- **Inputs:** Clean text streams from Phase 5.
- **Components:** Legal NER, Spatio-Temporal Parser (Duckling), Hard-ID Extractor.
- **Processing:** Token classification, regex validation (Luhn, E.164), temporal ISO 8601 interval calculation.
- **Storage:** PostgreSQL `extracted_entities`, `extracted_events`.
- **AI Models:** RoBERTa-Legal-NER, Duckling.
- **APIs:** `/api/v1/extract/entities`, `/api/v1/extract/identifiers`.
- **Outputs:** JSON list of validated entities, event tuples, and verified hard-identifiers.
- **Validation:** Precision > 95% on Hard-IDs; F1-score > 0.90 on legal named entities.
- **Security:** Absolute preservation of primary source citation for every extracted token.
- **Dependencies:** Phase 5.

Phase 7 — Structured Case Memory Assembly

- **Objective:** Populate relational case records and link evidence provenance.
- **Inputs:** Extracted entities and events from Phase 6.
- **Components:** Case Memory Synchronization Engine.
- **Processing:** Deduplication, entity canonicalization (resolving aliases), foreign-key relational linkage.
- **Storage:** PostgreSQL 16 (JSONB tables).
- **AI Models:** FastEmbed / BGE-Large (for entity deduplication candidate retrieval).
- **APIs:** `/api/v1/memory/sync`, `/api/v1/memory/entities`.
- **Outputs:** Canonical Case Memory Ledger.
- **Validation:** Zero dangling foreign keys; ACID transaction verification.
- **Security:** Row-Level Security (RLS) enforcing case boundaries.
- **Dependencies:** Phase 6.

Phase 8 — Case Knowledge Graph Construction

- **Objective:** Construct the property graph mapping all relationships, statements, and evidence citations.
- **Inputs:** Entities and relationships from Phase 7.
- **Components:** Neo4j Graph Synchronizer, Cypher Query Builder.
- **Processing:** Node creation, edge synthesis, provenance tagging on every graph property.
- **Storage:** Neo4j Enterprise.
- **AI Models:** None.
- **APIs:** `/api/v1/graph/query`, `/api/v1/graph/subgraph`.
- **Outputs:** Connected Case Knowledge Graph.
- **Validation:** Cypher query performance < 50ms on 4-hop relationship traversals.
- **Security:** Graph database authentication, isolation of multi-case graph namespaces.
- **Dependencies:** Phase 7.

Phase 9 — Timeline Engine & Contradiction Detection

- **Objective:** Construct chronological timelines and detect temporal/factual contradictions.
- **Inputs:** Events, statements, and medical findings from Knowledge Graph.
- **Components:** Interval Temporal Solver, Contradiction Detection Engine.
- **Processing:** Allen's interval logic evaluation; spatial transit velocity checking; statement vs forensic cross-matching.
- **Storage:** PostgreSQL `case_timeline`, `contradiction_records`.
- **AI Models:** None (Deterministic graph & relational algorithms).
- **APIs:** `/api/v1/timeline/build`, `/api/v1/contradictions/list`.
- **Outputs:** Chronological Event Timeline, Contradiction Matrix.
- **Validation:** 100% detection of synthetic spatio-temporal conflicts.
- **Security:** Contradictions presented without algorithmic resolution.
- **Dependencies:** Phase 8.

Phase 10 — Intuition Brain (Discovery & Pattern Layer)

- **Objective:** Deploy private air-gapped LLM to discover candidate insights, gaps, and anomalies.
- **Inputs:** Structured case memory subgraphs, timeline JSON slices.
- **Components:** Subgraph Serializer, vLLM Inference Engine.
- **Processing:** Context-bounded RAG; prompting for narrative tensions, timeline gaps, and missing witnesses.
- **Storage:** Internal Redis buffer (`candidate_insights_buffer`).
- **AI Models:** Llama-3-70B-Instruct (4-bit/8-bit quantized via vLLM).
- **APIs:** Internal gRPC `/inference/intuition/synthesize`.
- **Outputs:** `CandidateInsightPayload` JSON structures.
- **Validation:** Strict conformance to JSON schema; zero direct user exposure.
- **Security:** Air-gapped; prompt-injection testing; system prompts enforce Cardinal Anti-Suspect Rule.
- **Dependencies:** Phase 9.

Phase 11 — Discipline Brain (Verification Engine)

- **Objective:** Build the deterministic verification kernel to validate candidate insights.
- **Inputs:** `CandidateInsightPayload` from Phase 10, Primary Document Text from WORM storage.
- **Components:** Atomic Claim Decomposer, Text Span Locator, NLI Entailment Checker.
- **Processing:** Breaking insights into atomic claims; verifying document ID existence; exact string/fuzzy span matching.
- **Storage:** PostgreSQL `claim_verification_staging`.
- **AI Models:** DeBERTa-v3-large-MNLI (for entailment verification).
- **APIs:** Internal gRPC `/verify/claims`.
- **Outputs:** Verified, Unverified, and Contradictory Claim Records with Grounding Ratios.
- **Validation:** 100% rejection of synthetic hallucinated claims.
- **Security:** Cryptographic verification of source file hashes prior to span checking.
- **Dependencies:** Phase 10.

Phase 12 — Evidence Gate Implementation

- **Objective:** Implement the final gating mechanism controlling investigator-facing output.
- **Inputs:** Validated claim records from Phase 11.
- **Components:** Evidence Gatekeeper Service.
- **Processing:** Threshold evaluation (Grounding Ratio == 1.0); categorical routing.
- **Storage:** PostgreSQL `evidence_claim_mappings`.
- **AI Models:** None.
- **APIs:** `/api/v1/gate/evaluate`.
- **Outputs:** Verified Facts Board, Unverified Leads List, Rejection Logs.
- **Validation:** Zero unverified claims permitted into the Verified Facts stream.
- **Security:** Immutable gating decisions committed directly to audit logs.
- **Dependencies:** Phase 11.

Phase 13 — Deterministic Legal / Procedural Rule Engine

- **Objective:** Codify statutory criminal procedure checks (CrPC/BNSS).
- **Inputs:** Case metadata, stages, timestamps, uploaded document types.
- **Components:** Rule Evaluation Engine.
- **Processing:** Rule parsing, procedural deadline monitoring, compliance alert generation.
- **Storage:** PostgreSQL `legal_rules`, `procedural_compliance`.
- **AI Models:** None (100% deterministic rule engine).
- **APIs:** `/api/v1/rules/evaluate`, `/api/v1/rules/status`.
- **Outputs:** Procedural Compliance Badges, Statutory Breach Alerts.
- **Validation:** Unit tests modeling statutory time expirations (e.g., 24-hour magistrate reporting).
- **Security:** Rules are immutable and independently audited by legal counsel.
- **Dependencies:** Phase 3.

Phase 14 — Investigative Gap Engine

- **Objective:** Surface evidentiary omissions and generate evidence-backed next steps.
- **Inputs:** Case Knowledge Graph, Timeline, Verified Facts, Unverified Claims.
- **Components:** Graph Void Detector, Next Steps Formulator.
- **Processing:** Searching for unconnected entities, missing forensic lab tests, and uncorroborated alibis.
- **Storage:** PostgreSQL `investigative_gaps`, `suggested_next_steps`.
- **AI Models:** None.
- **APIs:** `/api/v1/gaps/list`, `/api/v1/next-steps/list`.
- **Outputs:** Evidentiary Gap Board, Actionable Investigation Steps.
- **Validation:** Accurate surfacing of missing procedural steps on historical mock cases.
- **Security:** Next steps strictly suggest evidence collection; never suggest arresting individuals.
- **Dependencies:** Phases 9, 12, 13.

Phase 15 — Cross-Case Hard-Identifier Linkage Engine

- **Objective:** Implement exact-match cross-case discovery restricted to machine-verifiable identifiers.
- **Inputs:** Hard-Identifiers (IMEI, Phone, Plate, Bank Acc) from active cases.
- **Components:** Hard-ID Indexer, Linkage Alert Engine, Firewall Kernel.
- **Processing:** Salted cryptographic hashing of identifiers; B-Tree exact lookups across cases; dropping non-verifiable attributes.
- **Storage:** PostgreSQL `cross_case_identifier_index`.
- **AI Models:** None.
- **APIs:** `/api/v1/linkage/query`, `/api/v1/linkage/alerts`.
- **Outputs:** Exact-Match Linkage Notifications.
- **Validation:** 0% false positives; 100% rejection of behavioral/MO queries.
- **Security:** Cross-case discovery shares only matching token and case registration number; no narrative leakage.
- **Dependencies:** Phase 6.

Phase 16 — Append-Only Reasoning Trail & Merkle Audit Service

- **Objective:** Build an immutable, tamper-evident audit ledger and statutory electronic certificate generator.
- **Inputs:** All API transactions, ingestion hashes, model prompts, outputs, and officer actions.
- **Components:** Merkle Tree Logger, Section 65B IEA / Section 63 BSA Certificate Generator.
- **Processing:** Computing cryptographic Merkle leaves, appending to tamper-evident ledger, generating digitally signed PDF certificates.
- **Storage:** Immudb Cryptographic Ledger.
- **AI Models:** None.
- **APIs:** `/api/v1/audit/logs`, `/api/v1/audit/export-certificate`.
- **Outputs:** Verifiable Audit Trail, Court-Admissible Electronic Record Certificates.
- **Validation:** Mathematical verification of Merkle root consistency upon ledger rebuild.
- **Security:** Write-only permissions; tamper detection triggers immediate administrator lockdown.
- **Dependencies:** Cross-cutting (Integrates across all phases).

Phase 17 — Investigator Dashboard & Visualization Suite

- **Objective:** Develop high-performance web interface for case exploration and decision-making.
- **Inputs:** API endpoints from all backend services.
- **Components:** Next.js Application, D3 Timeline Component, PDF Citation Viewer.
- **Processing:** UI state management, interactive graph rendering, side-by-side document span preview.
- **Storage:** Browser local storage for UI preferences.
- **AI Models:** None.
- **APIs:** REST / WebSocket APIs from FastAPI backend.
- **Outputs:** Interactive Web Dashboard.
- **Validation:** End-to-end user experience testing with investigating officers.
- **Security:** Content Security Policy (CSP), anti-clickjacking headers, secure session tokens.
- **Dependencies:** Phases 9, 12, 14, 15.

Phase 18 — Voice Assistant & Bilingual Audio Query Module

- **Objective:** Enable voice-driven queries and dictation in mixed Malayalam-English.
- **Inputs:** Investigator microphone audio streams.
- **Components:** WebRTC Gateway, Whisper ASR, Grounded Query Resolver.
- **Processing:** Real-time speech-to-text; intent extraction; case memory retrieval; evidence-grounded voice response synthesis.
- **Storage:** Temporary audio buffers, PostgreSQL `voice_query_logs`.
- **AI Models:** Faster-Whisper, Kokoro / Piper TTS.
- **APIs:** `/api/v1/voice/stream`, `/api/v1/voice/query`.
- **Outputs:** Spoken and textual evidence-backed answers with clickable citations.
- **Validation:** Response latency < 1.8 seconds; 100% citation compliance.
- **Security:** Audio streams encrypted via WebSockets over TLS; no audio retained permanently without user consent.
- **Dependencies:** Phases 5, 12.

Phase 19 — Case Diary & Report Generation Engine

- **Objective:** Generate statutory daily case diaries, evidence summaries, and gap audits.
- **Inputs:** Verified facts, officer actions, timeline entries, gap summaries.
- **Components:** Report Template Engine (WeasyPrint / ReportLab).
- **Processing:** Assembling verified records; clearly demarcating verified facts from unverified leads; compiling PDF dossiers.
- **Storage:** PostgreSQL `generated_reports`, MinIO PDF Vault.
- **AI Models:** None.
- **APIs:** `/api/v1/reports/case-diary`, `/api/v1/reports/gap-audit`.
- **Outputs:** Standardized, printable Case Diary PDFs.
- **Validation:** Legal format compliance with State Police Station Diary standards.
- **Security:** Digital watermarking, officer cryptographic signature embedding.
- **Dependencies:** Phases 12, 14, 16.

Phase 20 — Forensic Science Laboratory (FSL) Integration Layer

- **Objective:** Establish automated, secure API ingestion for official laboratory test results.
- **Inputs:** JSON/XML structured FSL payloads, signed digital lab certificates.
- **Components:** FSL Gateway Adapter, Certificate Validator.
- **Processing:** Inbound mTLS authentication, signature validation, parsing lab findings (toxicology, ballistics).
- **Storage:** MinIO FSL Vault, PostgreSQL `medical_forensic_facts`.
- **AI Models:** None.
- **APIs:** `/api/v1/integration/fsl/webhook`.
- **Outputs:** Verified Forensic Finding Nodes in Knowledge Graph.
- **Validation:** Verification of laboratory public key infrastructure (PKI) signatures.
- **Security:** Dedicated mTLS subnet; strict schema whitelisting.
- **Dependencies:** Phases 4, 7, 8.

Phase 21 — Comprehensive Red-Teaming, Bias & Adversarial Testing

- **Objective:** Subject the complete system to adversarial stress testing and jailbreak attempts.
- **Inputs:** Adversarial prompts, corrupted documents, synthetic contradictory cases.
- **Components:** Automated Red-Team Harness, Jailbreak Injection Suite.
- **Processing:** Testing whether LLM can be manipulated into naming/ranking suspects; verifying that zero unverified claims bypass the gate.
- **Storage:** PostgreSQL `test_audit_results`.
- **AI Models:** Multiple adversarial LLM attack agents.
- **APIs:** Full API surface.
- **Outputs:** Security Audit Report, Defect Remediation Log.
- **Validation:** 0 successful suspect predictions across 10,000 adversarial tests; 100% gatekeeper efficacy.
- **Security:** Verification of zero external cloud egress.
- **Dependencies:** All previous phases.

Phase 22 — Secure On-Premise Air-Gapped Deployment & Commissioning

- **Objective:** Deploy production cluster within secure, air-gapped police infrastructure.
- **Inputs:** Docker container images, Helm charts, bare-metal server infrastructure.
- **Components:** Kubernetes Cluster, NVIDIA GPU Operator, MinIO Distributed, Neo4j Cluster, PostgreSQL Cluster.
- **Processing:** Infrastructure orchestration, air-gapped container loading, KMS key initialization.
- **Storage:** High-availability distributed storage arrays.
- **AI Models:** Local LLM & ASR model weight validation via SHA-256 tokens.
- **APIs:** Production Gateway Endpoints.
- **Outputs:** Live, operational Sakshi System.
- **Validation:** High availability failover drills, disaster recovery validation.
- **Security:** Physical air-gap verification; external network interface disconnection; OS kernel hardening.
- **Dependencies:** Phase 21.

SECTION R — Phase-to-Component Mapping Table

Phase	Module Name	Input Data	Core Processing	Output Data	Primary Datastore	Consuming Module
Phase 1	Policy Engine	Statutory Acts (CrPC/BNSS)	Rule formalization, policy codification	JSON Schema Policies	Git / JSON Registry	Phase 2, Phase 3
Phase 2	DB Modeler	Policy Specs	Relational & Graph schema design	DDL Scripts, Storage Config	PostgreSQL / Neo4j	Phase 3, Phase 4

Phase	Module Name	Input Data	Core Processing	Output Data	Primary Datastore	Consuming Module
Phase 3	Case Manager	FIR Metadata, IO Credentials	Case creation, RBAC validation	Active Case Workspace	PostgreSQL <code>cases</code>	Phase 4, Phase 17
Phase 4	Ingestion Engine	Binary Evidence Files	Dual-hashing (SHA-256/SHA-3), ClamAV	WORM URI, Hash Tokens	MinIO WORM Vault	Phase 5, Phase 16
Phase 5	OCR & ASR	Scanned PDFs, Voice Notes	Deskew, layout OCR, Malayalam ASR	Normalized Text Streams	PostgreSQL <code>processed_docs</code>	Phase 6
Phase 6	NLP & Extraction	Text Streams	Legal NER, ISO 8601, Hard-ID regex	Entities, Events, Hard-IDs	PostgreSQL <code>entities</code>	Phase 7, Phase 15
Phase 7	Case Memory Sync	Entities & Events	Canonicalization, deduplication	Structured Case State	PostgreSQL 16 (JSONB)	Phase 8, Phase 9
Phase 8	Graph Builder	Normalized Entities/Events	Node/edge creation, provenance tag	Property Knowledge Graph	Neo4j Enterprise	Phase 9, Phase 10
Phase 9	Timeline & Contradiction	Graph Nodes, Timestamps	Allen's algebra, velocity check	Chronological Timeline	PostgreSQL <code>timeline</code>	Phase 10, Phase 14
Phase 10	Intuition Brain	Subgraphs, Timeline Slices	LLM context reasoning, anomaly search	<code>CandidateInsightPayload</code>	In-Memory Buffer	Phase 11
Phase 11	Discipline Brain	Candidate Insights, Raw Docs	Claim decomposition, span match	Grounded Claim Records	PostgreSQL <code>staging</code>	Phase 12
Phase 12	Evidence Gate	Grounded Claim Records	Threshold check (Grounding == 1.0)	Verified & Unverified Lists	PostgreSQL <code>claim_map</code>	Phase 14, Phase 17
Phase 13	Legal Rule Engine	Case State, Timestamps	Procedural deadline checking	Statutory Alerts & Status	PostgreSQL <code>compliance</code>	Phase 14, Phase 19
Phase 14	Gap Engine	Graph, Timeline, Gate Claims	Omission detection, next steps	Gaps & Next Steps	PostgreSQL <code>gaps</code>	Phase 17, Phase 19
Phase 15	Hard-ID Linker	Verified Hard-IDs	B-Tree exact lookups, salt hashing	Cross-Case Match Alerts	PostgreSQL <code>id_index</code>	Phase 17

Phase	Module Name	Input Data	Core Processing	Output Data	Primary Datastore	Consuming Module
Phase 16	Merkle Audit	System events, Model I/O	Merkle tree hashing, Sec 65B cert	Immutable Audit Ledger	Immudb WORM Ledger	Phase 17, Phase 19
Phase 17	Investigator UI	API Streams	Rendering interactive views	Visualized Dashboards	Browser Local Storage	Investigating Officer
Phase 18	Voice Assistant	Audio Queries	Streaming ASR, grounded retrieval	Voice Answers + Citations	PostgreSQL voice_logs	Investigating Officer
Phase 19	Case Diary Engine	Verified Facts, IO Actions	Compiling statutory case diary	Printable Case Diary PDF	MinIO Report Vault	Investigating Officer
Phase 20	FSL Gateway	Digital Lab Certificates	mTLS validation, schema parsing	Verified Lab Fact Nodes	Neo4j / PostgreSQL	Phase 8, Phase 9
Phase 21	Red Team Suite	Adversarial Prompts	Jailbreak testing, bias evaluation	Audit Sign-off Dossier	PostgreSQL test_logs	Phase 22
Phase 22	Deployment	Container Images	Air-gapped orchestration	Operational Cluster	Production Hardware	End Users

SECTION S — Database & Data Storage Architecture

Sakshi segregates case data across 10 specialized storage engines:

- DS-1: Raw Evidence Object Storage (MinIO Enterprise):** WORM-compliant S3 storage holding raw unmodified evidentiary binaries (scanned PDFs, photographs, audio recordings, CDR spreadsheets). Configured with Object Lock in Compliance Mode.
- DS-2: Processed Evidence Store (MinIO / PostgreSQL):** Cleaned text files, normalized CSV tables, OCR bounding box files, and ASR word-level timestamp transcriptions.
- DS-3: Structured Case Database (PostgreSQL 16):** Fully normalized relational schema storing case metadata, officer profiles, canonical entities, timeline intervals, and legal compliance states.
- DS-4: Case Knowledge Graph (Neo4j Enterprise):** Property graph housing all entities, relationships, assertions, and evidentiary provenance citations.
- DS-5: Vector Semantic Retrieval Store (Qdrant):** Dense embeddings of document paragraphs used exclusively for semantic candidate retrieval. Strictly partitioned by Case ID.
- DS-6: Evidence-Claim Mapping Store (PostgreSQL):** Maps every atomic claim to its primary source evidence ID, text span coordinates, grounding ratio, and verification status.
- DS-7: Append-Only Reasoning & Audit Store (Immudb):** Cryptographic Merkle tree ledger recording all system events, AI prompts, model outputs, verification queries, and investigator actions.

- 9/29/26, 10:07 PM
- SAKSHI: Master Architecture & Engineering Specification
8. **DS-8: Identity & Access Management Store (Keycloak PostgreSQL):** Manages user credentials, role definitions, attribute-based access policies, and MFA registrations.

9. **DS-9: Search Index (Elasticsearch / OpenSearch):** Full-text search index across all OCR transcripts and case notes, supporting boolean and proximity searches.

10. **DS-10: Cold Backup & Disaster Recovery Storage:** Encrypted, off-site, air-gapped physical tape or disconnected disk backups updated via daily snapshots.
-
-

SECTION T — Deployment Architecture

Sakshi is deployed in an **Air-Gapped On-Premise Topology**:



✖ PHYSICAL AIR GAP: NO EXTERNAL INTERNET CONNECTION

SECTION U — The 20 Architectural Diagrams & Technical Explanations

FIGURE 1 — System Context Diagram



```
class EXT,IO,FSL_EXT,DOCS,DIGI,TEL ext;
class SYS,CORE sys;
class OUTS,VER_FACTS,CONTR_VIEW,GAP_VIEW,CASE_DIARY,AUDIT_CERT out;
```

Technical Explanation: Figure 1

- **Description:** Establishes the macro-system perimeter. External entities feed multi-modal documents, lab data, and voice commands into the secure boundary. Sakshi processes these inputs and generates strictly verified outputs, contradiction flags, procedural gap alerts, and electronic record certificates.
- **Input:** FIRs, witness testimonies, digital CDRs, forensic reports, and investigator interactions.
- **Processing:** Encapsulation, verification, cognitive reasoning, and evidence gating.
- **Output:** Verified facts, contradiction matrices, gap boards, and statutory case diaries.
- **Data Movement:** Boundary data enters through authenticated gateways; outputs flow to authorized web and export interfaces.
- **Technology:** Kong API Gateway, TLS 1.3, Next.js, FastAPI.

FIGURE 2 — Overall Layered System Architecture

```
flowchart TB
    subgraph LA["Layer A: User / Investigator Layer"]
        UI_DASH["Investigator Web Dashboard"]
        UI_VOICE["Voice Assistant & Dictation Console"]
        UI_DIARY["Case Diary & Report Interface"]
        UI_BOARD["Hypothesis Board & Gap Explorer"]
    end

    subgraph LB["Layer B: Application Services Layer"]
        SRV_CASE["Case Management Service"]
        SRV_VOICE["Voice Query Processing Service"]
        SRV_GAP["Investigative Gap & Next Steps Service"]
        SRV_LINK["Hard-ID Cross-Case Linkage Service"]
    end

    subgraph LC["Layer C: Two-Brain Intelligence Tier"]
        subgraph IB_S["Intuition Brain (LLM Sandbox)"]
            IB_CORE["vLLM Llama-3-70B\n(Pattern & Anomaly Discovery)"]
        end
        FIREWALL["Mandatory Informational Firewall"]
        subgraph DB_S["Discipline Brain (Auditor)"]
            DB_CORE["Claim Decomposer & Grounding Engine"]
            GATE_CORE["Evidence Gate (Grounding = 1.0)"]
        end
        IB_CORE --> FIREWALL --> DB_CORE --> GATE_CORE
    end
```

```
subgraph LD["Layer D: Structured Case Memory Tier"]
    DB_REL[("PostgreSQL 16 (Case DB)")]
    DB_GRAPH[("Neo4j (Knowledge Graph)")]
    DB_TIME[("Chronological Timeline Engine")]
    DB_VEC[("Qdrant Vector Store")]
end

subgraph LE["Layer E: Evidence Processing & Extraction"]
    PROC_OCR[("OCR Engine (Tesseract/Paddle)")]
    PROC_ASR[("Bilingual Whisper ASR")]
    PROC_NLP[("Legal NER & ISO 8601 Parser")]
    PROC_RULE[("Deterministic Legal Rule Engine")]
end

subgraph LF["Layer F: Evidence Ingestion & WORM Vault"]
    ING_GATE[("Ingestion Controller & ClamAV")]
    ING_HASH[("SHA-256 & SHA-3 Dual Hasher")]
    VAULT_S3[("MinIO WORM Object Vault")]
end

subgraph CROSS["Cross-Cutting Security & Audit"]
    AUDIT_LOG[("ImmuDB Append-Only Merkle Ledger")]
    SEC_KMS[("Keycloak IAM & Hardware KMS (AES-256)")]
end

LA → LB
LB → LC
LC → LD
LD → LE
LE → LF

CROSS -. LA
CROSS -. LC
CROSS -. LD
CROSS -. LF

classDef layer fill:#1e1b4b,stroke:#818cf8,stroke-width:1.5px,color:#fff;
classDef gate fill:#064e3b,stroke:#34d399,stroke-width:2px,color:#fff;
classDef store fill:#312e81,stroke:#a5b4fc,stroke-width:1px,color:#fff;
class LA,LB,LE,LF layer;
class LC,GATE_CORE,DB_CORE gate;
class LD,VAULT_S3,AUDIT_LOG,DB_REL,DB_GRAPH,DB_VEC store;
```

Technical Explanation: Figure 2

- **Description:** Complete six-layer architectural stack with cross-cutting security and Merkle audit trails. Illustrates the strict upward flow from immutable raw storage to presentation.

- **Input:** Multi-modal investigation artifacts.
- **Processing:** Layer-by-layer enrichment: Ingestion → Hashing → Extraction → Graph Representation → Two-Brain Reasoning → Presentation.
- **Output:** Decoupled, highly available operational microservices.
- **Data Movement:** gRPC between intelligence services; REST/WebSocket for UI; immutable writes to storage.
- **Technology:** Docker, Kubernetes, MinIO, PostgreSQL, Neo4j, vLLM, Next.js.

FIGURE 3 — Complete End-to-End Investigation Workflow



Technical Explanation: Figure 3

- **Description:** Closed-loop operational workflow tracing an inquiry from the initial unnatural death report through continuous evidence gathering cycles.
- **Input:** Initial reporting details followed by cyclical field discoveries.

- **Processing:** Sequential verification, cognitive reasoning, human deliberation, and iterative case memory updates.
- **Output:** Progressively refined, verified case intelligence.
- **Data Movement:** Linear forward execution with an asynchronous feedback loop reconnecting field actions to raw ingestion.
- **Technology:** Celery task queues, RabbitMQ, PostgreSQL event triggers.

FIGURE 4 — Level-0 Data Flow Diagram (Context DFD)



Technical Explanation: Figure 4

- **Description:** Level-0 DFD showing environmental data exchange across external boundaries. Emphasizes that Sakshi acts as an evidentiary intermediary between data custodians and the human investigator.
- **Input:** Documentary, digital, and voice streams.
- **Processing:** High-level ingestion, verification, gap detection, and reporting.
- **Output:** Verified facts, contradiction alerts, and electronic record certificates.
- **Data Movement:** Boundary crossing via secured APIs.
- **Technology:** HTTPS, mTLS, WSS.

FIGURE 5 — Level-1 Data Flow Diagram (Subsystem Deconstruction)

```
flowchart TD
    I0["Investigator"] -->|1. Evidence Binary| P1["1.0 Ingestion & Hashing"]
    P1 -->|Raw Byte Stream| D1["DS-1: Raw WORM Vault"]
    P1 -->|Checksum & Metadata| D7["DS-7: Append-Only Audit"]

    D1 -->|Read File| P2["2.0 OCR & Extraction"]
    P2 -->|Structured Entities & IDs| D3["DS-3: Case Database"]
    P2 -->|Graph Tuples| D4["DS-4: Knowledge Graph"]

    D3 & D4 -->|Case Context| P3["3.0 Legal Rule Check"]
    P3 -->|Compliance Alerts| D3

    D4 -->|Graph Slices| P4["4.0 Intuition Brain (LLM)"]
    P4 -->|Candidate Insights| P5["5.0 Discipline Brain"]

    D1 & D3 & D4 -.→|Grounding Query| P5
    P5 -->|Verification Verdict| P6["6.0 Evidence Gate"]

    P6 -->|Verified Facts / Gaps| D6["DS-6: Claim Mapping"]
    D6 -->|Display Data| P7["7.0 Investigator Interface"]
    P7 -->|Inspect & Act| I0
    P7 -->|Log Actions| D7

    classDef proc fill:#0f172a,stroke:#38bdf8,stroke-width:1.5px,color:#fff;
    classDef store fill:#1e1b4b,stroke:#818cf8,stroke-width:1.5px,color:#fff;
    class P1,P2,P3,P4,P5,P6,P7 proc;
    class D1,D3,D4,D6,D7 store;
```

Technical Explanation: Figure 5

- **Description:** Level-1 DFD detailing internal data transformations across 7 primary functional processes and 5 major datastores.
- **Input:** Raw evidence, case context, investigator actions.
- **Processing:** Ingestion, feature extraction, legal compliance checking, two-brain reasoning, gating, and UI presentation.
- **Output:** Categorized factual claims, gap reports, and audit trails.
- **Data Movement:** Controlled read/write interactions across relational, graph, object, and ledger stores.
- **Technology:** FastAPI, Celery, Neo4j, PostgreSQL, Immudb.

FIGURE 6 — Evidence Ingestion Pipeline

```
flowchart LR
    subgraph INTAKE["1. Intake & Validation"]
        F_IN["Inbound File"] --> SIGN["MIME & Magic Byte Check"]
        SIGN --> AV["ClamAV Antivirus Scan"]
    end
```

```
end

subgraph CRYPTO["2. Integrity & Hashing"]
    AV → HASH["Dual-Hasher\n(SHA-256 & SHA-3-512)"]
    HASH → WORM["MinIO WORM Storage\n(Object Lock Compliance)"]
    HASH → MERKLE["Commit Hash to Immudb"]
end

subgraph EXTRACT["3. Processing & Transcription"]
    WORM → ROUTE{"File Type?"}
    ROUTE -- PDF / Image → OCR["PaddleOCR / Tesseract\n(Deskew + Binarization)"]
    ROUTE -- Audio → ASR["Faster-Whisper\n(Malayalam + English ASR)"]
    ROUTE -- Tabular / CDR → PARSE["Structured Table Parser\n(Cell ID & IMSI Mapper)"]
end

subgraph OUT_DATA["4. Normalized Output"]
    OCR & ASR & PARSE → NORM["UTF-8 Stream + Coordinate\n& Timestamp Spans"]
end

classDef intake fill:#0f172a,stroke:#38bdf8,stroke-width:1.5px,color:#fff;
classDef crypto fill:#064e3b,stroke:#34d399,stroke-width:1.5px,color:#fff;
classDef proc fill:#311b92,stroke:#818cf8,stroke-width:1.5px,color:#fff;
class INTAKE,F_IN,SIGN,AV intake;
class CRYPTO,HASH,WORM,MERKLE crypto;
class EXTRACT,ROUTE,OCR,ASR,PARSE,OUT_DATA,NORM proc;
```

Technical Explanation: Figure 6

- **Description:** Details the intake, cryptographic encapsulation, and multi-modal transcription pipeline. Enforces strict virus scanning and dual-hashing before running OCR or speech-to-text.
- **Input:** Multi-format police files (PDF scans, JPEG panchnamas, WAV/MP3 audio, Excel CDRs).
- **Processing:** Signature verification, streaming hashing, WORM persistence, bilingual OCR, and speech transcription.
- **Output:** Normalized UTF-8 text streams with precise spatial and temporal bounding metadata.
- **Data Movement:** Binary streams pass into memory buffers, flush to WORM storage, and route to async worker pools.
- **Technology:** Python hashlib, ClamAV, MinIO Object Lock, PaddleOCR, Faster-Whisper.

FIGURE 7 — Structured Case Memory / Knowledge Graph Schema

```
classDiagram
    class Person {
        +UUID person_id
        +String name
        +List aliases
        +String legal_status
    }
```

```

}

class Statement {
    +UUID statement_id
    +String section_code
    +DateTime recorded_at
    +String text_content
}

class Event {
    +UUID event_id
    +DateTime interval_start
    +DateTime interval_end
    +String location
    +String description
}

class HardIdentifier {
    +String id_type
    +String normalized_value
    +Boolean machine_verified
}

class EvidenceItem {
    +UUID evidence_id
    +String sha256_hash
    +String storage_uri
    +DateTime collected_at
}

class MedicalFinding {
    +UUID finding_id
    +String cause_of_death
    +DateTime pmi_start
    +DateTime pmi_end
    +Boolean antemortem
}

class Contradiction {
    +UUID contradiction_id
    +String nature
    +String status
}

Person "1" → "*" Statement : PROVIDES
Person "1" → "*" HardIdentifier : HAS_IDENTIFIER
Statement "1" → "*" Event : RECOUNTS
EvidenceItem "1" → "1" Statement : EMBODIES
EvidenceItem "1" → "*" MedicalFinding : CONTAINS
Event "*" → "1" EvidenceItem : PROVEN_BY
Event "1" → "1" Contradiction : INVOLVED_IN
MedicalFinding "1" → "1" Contradiction : REFUTES_EVENT

```

Technical Explanation: Figure 7

- **Description:** UML class diagram mapping the entities, relationships, attributes, and provenance links within the Neo4j Knowledge Graph.
- **Input:** Extracted entities, events, identifiers, and medical findings.
- **Processing:** Schema validation and graph edge construction with mandatory provenance properties.
- **Output:** Connected Property Graph single-source-of-truth.
- **Data Movement:** Extracted JSON tuples are written via parameterized Cypher transactions.
- **Technology:** Neo4j 5.x, Cypher, Pydantic.

FIGURE 8 — Timeline Construction & Contradiction Detection



```
class INP,E1,E2,E3,E4,E5 inp;
class ENGINE,NORM,VEL,ALLEN eng;
class CONTR_DETECT,CLASH1,CLASH2 alert;
class OUT_TL,TL_VIEW,MATRIX out;
```

Technical Explanation: Figure 8

- **Description:** Details chronological timeline assembly and contradiction detection using spatio-temporal velocity checks and Allen's Interval Algebra.
- **Input:** Timestamps and locations from statements, CDRs, CCTV logs, and post-mortems.
- **Processing:** Interval normalization, transit velocity calculation, and conflict identification.
- **Output:** Continuous interactive timeline and highlighted contradiction matrix.
- **Data Movement:** Extracted event tuples pass through algebra engines; flagged conflicts write to PostgreSQL contradiction tables.
- **Technology:** Python, Duckling, NetworkX, PostgreSQL.

FIGURE 9 — Two-Brain Architecture & Firewall Boundary

```
flowchart TD
    subgraph MEM["Authoritative Case Memory"]
        KG["Neo4j Knowledge Graph"]
        RAW["WORM Evidence Vault"]
    end

    subgraph IB["INTUITION BRAIN (Exploration Engine)"]
        CTX["Context Assembler"]
        LLM["Air-Gapped vLLM (Llama-3-70B)\nSystem Prompt: Spot Omissions, Tensions, & Gaps"]
        CAND["CandidateInsightPayload (JSON)"]
        CTX --> LLM --> CAND
    end

    subgraph WALL["MANDATORY INFORMATIONAL FIREWALL"]
        BLOCK["Physical Barrier: Intuition Brain has NO Route to UI\nRaw LLM text CANNOT reach Investigator Console"]
    end

    subgraph DB["DISCIPLINE BRAIN (Auditing Engine)"]
        DEC["Atomic Claim Decomposer"]
        VERIF["Deterministic Evidence Verifier\n- Hash existence check\n- Exact text span match\n- NLI entailment sc"]
        DEC --> VERIF
    end

    subgraph GATE["THE EVIDENCE GATE"]
        EVAL["Grounding Ratio = 1.0\n& Consistent?"]
        V_OK["VERIFIED FACT"]
    end
```

```
V_UN["UNVERIFIED LEAD"]
V_CT["CONTRADICTION"]

end

subgraph UI["Investigator Interface"]
    OUT_UI["Investigator Console"]
end

KG → CTX
CAND → BLOCK
BLOCK → DEC
RAW -.→|Cross-Verify Spans| VERIF
VERIF → EVAL
EVAL -- Yes → V_OK
EVAL -- Incomplete → V_UN
EVAL -- Clash → V_CT
V_OK & V_UN & V_CT → OUT_UI

classDef brain fill:#311b92,stroke:#818cf8,stroke-width:2px,color:#fff;
classDef wall fill:#7f1d1d,stroke:#f87171,stroke-width:2px,color:#fff;
classDef gate fill:#064e3b,stroke:#34d399,stroke-width:2px,color:#fff;
class IB,CTX,LLM,CAND brain;
class WALL,BLOCK wall;
class DB,DEC,VERIF,GATE,EVAL,V_OK,V_UN,V_CT gate;
```

Technical Explanation: Figure 9

- **Description:** Illustrates the asymmetric Two-Brain relationship. Shows the complete containment of the Intuition Brain behind an informational firewall, requiring deterministic proof from the Discipline Brain before reaching the investigator.
- **Input:** Case Knowledge Graph subgraphs and raw document text.
- **Processing:** Generative pattern discovery, atomic decomposition, and multi-point evidentiary verification.
- **Output:** Categorized verified facts, unverified leads, and contradictions.
- **Data Movement:** Strict unidirectional flow from LLM to Auditor via internal JSON buffers; zero direct connection to presentation layer.
- **Technology:** vLLM, DeBERTa-v3-large-MNLI, Pydantic, gRPC.

FIGURE 10 — Discipline Brain & Evidence Gate

```
flowchart TD
    IN_CAND["Candidate Claim: 'Witness stated weapon was thrown in river'"] --> S1["Decompose into Atomic Propositions"]
    S1 --> S2["Extract Cited Document ID: EVD-DOC-012, Page: 4"]
    S2 --> Q1["Does Document ID exist\nin Raw Vault?"]
    Q1 -- No --> R_REJ["REJECT: Hallucinated Document Citation"]
```

```
Q1 -- Yes --> Q2{"Does SHA-256 Hash Match\nRegistered Ingestion Hash?"}

Q2 -- No --> R_ALERT["SECURITY ALERT: Document Tamper Detected"]
Q2 -- Yes --> Q3{"Locate Exact Text Span in\nDocument OCR Stream?"}

Q3 -- No --> R_UNV1["MARK UNVERIFIED: Cited Span Missing"]
Q3 -- Yes --> Q4{"Does Semantic Content Entail\nAtomic Proposition?"}

Q4 -- No --> R_UNV2["MARK UNVERIFIED: Weak/Extrapolated Inference"]
Q4 -- Yes --> Q5{"Does Claim Conflict with Confirmed\nForensic or Medical Ground Truth?"}

Q5 -- Yes --> R_CONTR["MARK CONTRADICTION: Clashes with Forensic Fact"]
Q5 -- No --> R_VER["MARK VERIFIED: Grounding Ratio = 1.0"]

R_VER --> OUT_VER["Verified Facts Board (Click-to-Source Enabled)"]
R_UNV1 & R_UNV2 --> OUT_UNV["Unverified Leads / Gap Analysis Board"]
R_CONTR --> OUT_CONTR["Contradiction Resolution Matrix"]

classDef pass fill:#064e3b,stroke:#34d399,stroke-width:1.5px,color:#fff;
classDef fail fill:#7f1d1d,stroke:#f87171,stroke-width:1.5px,color:#fff;
classDef warn fill:#78350f,stroke:#fbbf24,stroke-width:1.5px,color:#fff;
class R_VER,OUT_VER pass;
class R_REJ,R_ALERT fail;
class R_UNV1,R_UNV2,R_CONTR,OUT_UNV,OUT_CONTR warn;
```

Technical Explanation: Figure 10

- **Description:** Algorithmic flowchart of the Evidence Gatekeeper evaluating an atomic candidate proposition against primary evidentiary sources.
- **Input:** Atomic proposition with claimed document ID, page, and text span.
- **Processing:** Existence check, hash validation, span location, NLI entailment scoring, and conflict checking.
- **Output:** Categorical gate verdict determining target investigator dashboard routing.
- **Data Movement:** Read lookups against WORM storage and PostgreSQL document tables; writes to `evidence_claim_mappings`.
- **Technology:** Python, Pydantic, RapidFuzz, DeBERTa-v3-large-MNLI.

FIGURE 11 — Investigative Gap Detection Architecture

```
flowchart TD
    subgraph INPUTS["Case Graph & Timeline Inputs"]
        KG_STATE["Knowledge Graph Entities & Relationships"]
        TL_STATE["Chronological Timeline Slices"]
        VER_STATE["Verified Facts & Unverified Claims"]
    end
```

```
subgraph DETECTOR["Investigative Gap Detection Engine"]
    G1["Alibi Corroboration Checker\n(Scans for unverified presence during PMI)"]
    G2["Procedural SOP Checker\n(Scans for missing inquest forms, FSL receipts)"]
    G3["Timeline Void Checker\n(Scans for unaccounted intervals > 60 mins)"]
    G4["Telecom Corroboration Checker\n(Scans for calls lacking tower records)"]
end

subgraph OUTPUT_GAPS["Identified Evidentiary Gaps"]
    GAP1["Gap: No tower CDR obtained for suspect alibi phone"]
    GAP2["Gap: Viscera FSL dispatch receipt missing in vault"]
    GAP3["Gap: Unaccounted 90-minute window prior to PMI"]
end

subgraph ACTIONS["Evidence-Backed Next Steps Generator"]
    STEP1["Suggested Action: Summon Tower CDR via Sec 91 CrPC/BNSS notice"]
    STEP2["Suggested Action: Requisition State FSL for Viscera chemical report"]
    STEP3["Suggested Action: Inquire commercial CCTV along transit corridor"]
end

INPUTS → DETECTOR
G1 → GAP1
G2 → GAP2
G3 → GAP3

GAP1 → STEP1
GAP2 → STEP2
GAP3 → STEP3

ACTIONS → DASH["Investigator Gap Board & Action Console"]

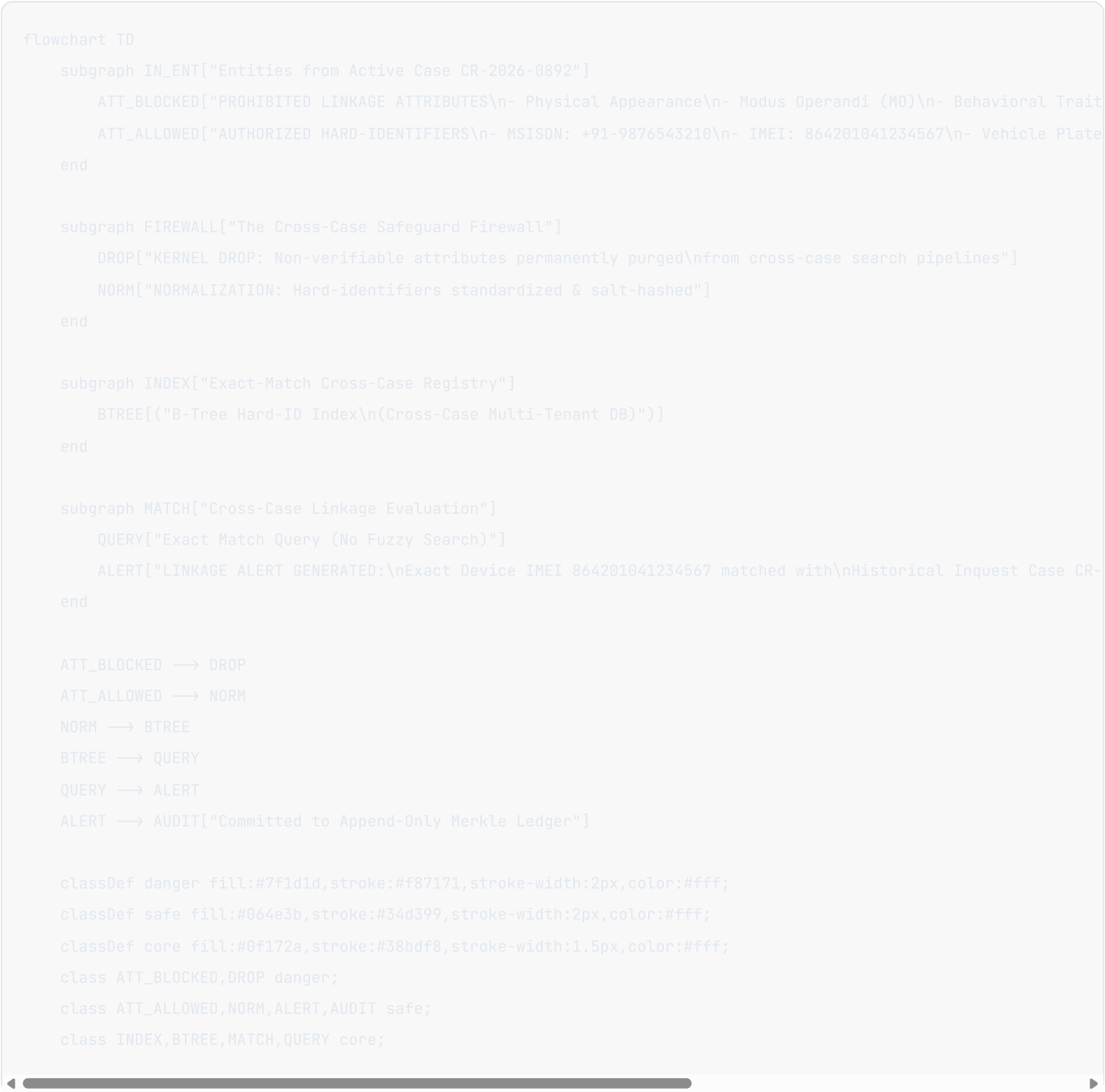
classDef inp fill:#1e293b,stroke:#64748b,stroke-width:1px,color:#fff;
classDef eng fill:#0f172a,stroke:#38bdf8,stroke-width:1.5px,color:#fff;
classDef gap fill:#78350f,stroke:#fbbf24,stroke-width:1.5px,color:#fff;
classDef act fill:#064e3b,stroke:#34d399,stroke-width:1.5px,color:#fff;
class INPUTS,KG_STATE,TL_STATE,VER_STATE inp;
class DETECTOR,G1,G2,G3,G4 eng;
class OUTPUT_GAPS,GAP1,GAP2,GAP3 gap;
class ACTIONS,STEP1,STEP2,STEP3,DASH act;
```

Technical Explanation: Figure 11

- **Description:** Details the detection of evidentiary voids and procedural lapses, translating identified gaps into lawful, actionable investigation steps.
- **Input:** Case Knowledge Graph, verified timeline, and procedural rules.
- **Processing:** Algorithmic gap detection (alibi voids, missing SOP items, timeline blackouts).
- **Output:** Actionable next-step recommendations for the investigator.

- **Data Movement:** Graph queries feed gap detection workers; outputs populate the investigator gap board.
- **Technology:** Neo4j Cypher, Python, PostgreSQL.

FIGURE 12 — Cross-Case Hard-Identifier Linkage Engine



Technical Explanation: Figure 12

- **Description:** Details the Hard-Identifier Linkage Safeguard. Demonstrates the kernel-level blocking of subjective/behavioral profiling attributes, permitting linkage only through exact machine-verifiable tokens.
- **Input:** Extracted entity attributes from active and historical case files.
- **Processing:** Attribute firewalling, regex validation, normalization, and exact B-Tree lookups.

- **Output:** Machine-verifiable match notifications.
- **Data Movement:** Normalized tokens query an isolated multi-tenant index; matches log to the Merkle audit trail.
- **Technology:** Python-phonenumbers, Luhn algorithm, PostgreSQL B-Tree indexes, Immudb.

FIGURE 13 — Legal / Procedural Rule Engine (Deterministic Non-LLM)



Technical Explanation: Figure 13

- **Description:** Illustrates the deterministic, non-LLM statutory rule engine enforcing compliance with criminal procedure codes and evidence preservation deadlines.
- **Input:** Case milestones, document timestamps, and procedural status.
- **Processing:** Rule evaluation against statutory legal conditions.
- **Output:** Compliance badges, active deadline timers, and statutory breach alerts.
- **Data Movement:** Event triggers evaluate state against rule tables; updates write to compliance registries.
- **Technology:** Python, Open Policy Agent (OPA) / custom Python rule engine, PostgreSQL.

FIGURE 14 — Reasoning & Court Audit Trail Architecture



Technical Explanation: Figure 14

- **Description:** Details the cryptographic Merkle tree audit architecture ensuring court-admissibility under statutory electronic evidence laws.
- **Input:** System operational transactions, AI inputs/outputs, and investigator decisions.
- **Processing:** Hash-chaining and Merkle root calculation.
- **Output:** Tamper-evident ledger logs and automated electronic evidence certificates.
- **Data Movement:** Events append sequentially to Immudb; export services read cryptographic proofs.
- **Technology:** Immudb, Python cryptography library, ReportLab.

FIGURE 15 — Human-in-the-Loop Architecture



Technical Explanation: Figure 15

- **Description:** Demonstrates human primacy in the investigation loop. Shows that AI outputs serve solely as decision support, while all statutory decisions, field inquiries, and justifications remain with the investigator.
- **Input:** Gated intelligence outputs.
- **Processing:** Investigator review, deliberation, action selection, and field execution.
- **Output:** Statutory field inquiries and supplementary evidentiary inputs.
- **Data Movement:** UI interaction triggers audit logs; field findings re-enter via ingestion.
- **Technology:** React, Next.js, FastAPI, PostgreSQL.

FIGURE 16 — Voice + Multilingual Processing Architecture



Technical Explanation: Figure 16

- **Description:** Architecture of the voice-enabled assistant handling mixed Malayalam-English speech, preserving original language recordings while supporting grounded query resolution.
- **Input:** Spoken audio streams from field officers or witness recordings.
- **Processing:** Voice activity detection, code-switching transcription, grounded query execution, and text-to-speech synthesis.
- **Output:** Transcribed bilingual text streams and spoken evidence-backed answers with citations.
- **Data Movement:** Real-time audio chunks pass via WebSockets to local Whisper instances; transcribed queries execute against case memory.
- **Technology:** WebRTC, Silero VAD, Faster-Whisper, Kokoro TTS.

FIGURE 17 — Forensic Science Laboratory (FSL) Integration Architecture



Technical Explanation: Figure 17

- **Description:** Details automated integration with official Forensic Science Laboratories. Shows secure mTLS intake, digital signature validation, and immediate contradiction re-checking upon lab result arrival.
- **Input:** Digitally signed laboratory forensic reports (toxicology, ballistics, DNA).
- **Processing:** PKI signature verification, schema validation, WORM persistence, and knowledge graph ingestion.
- **Output:** Authenticated forensic nodes in the case graph triggering automated re-evaluation of witness claims.
- **Data Movement:** Inbound HTTPS POST over dedicated mTLS subnet; event-driven worker updates graph memory.
- **Technology:** Mutual TLS, X.509 certificates, FastAPI, MinIO, Neo4j.

FIGURE 18 — Multi-Tier Defense-in-Depth Security Architecture



```
MFA["Hardware FIDO2 Token + Police Biometric MFA"]
ABAC["Attribute-Based Access Control (Keycloak)"]
VPN → MFA → ABAC

end

subgraph T2["Tier 2: Cryptographic Isolation & Storage Security"]
    KMS["Hardware Security Module (HSM) / Dedicated KMS"]
    ENC_REST["AES-256-GCM Storage Encryption (Unique Key Per Case)"]
    WORM_LOCK["MinIO S3 Compliance Mode (Object Lock)"]
    KMS → ENC_REST → WORM_LOCK
end

subgraph T3["Tier 3: Air-Gapped Cognitive Enclave"]
    AIRGAP["Physical / Logical Air-Gap: No Internet Egress"]
    LOCAL_GPU["Dedicated Local GPU Cluster (vLLM Engine)"]
    EPHEM["Stateless Inference Memory (Zero Retention of Prompts)"]
    AIRGAP → LOCAL_GPU → EPHEM
end

subgraph T4["Tier 4: Immutability & Audit Assurance"]
    MERKLE_T["Immudb Append-Only Merkle Tree Ledger"]
    TAMPER_AL["Real-time Hash Chain Validation & Tamper Alarms"]
    MERKLE_T → TAMPER_AL
end

T1 → T2
T2 → T3
T3 → T4

classDef t1 fill:#0f172a,stroke:#38bdf8,stroke-width:1.5px,color:#fff;
classDef t2 fill:#1e1b4b,stroke:#818cf8,stroke-width:1.5px,color:#fff;
classDef t3 fill:#311042,stroke:#a21caf,stroke-width:1.5px,color:#fff;
classDef t4 fill:#064e3b,stroke:#34d399,stroke-width:1.5px,color:#fff;
class T1,VPN,MFA,ABAC t1;
class T2,KMS,ENC_REST,WORM_LOCK t2;
class T3,AIRGAP,LOCAL_GPU,EPHEM t3;
class T4,MERKLE_T,TAMPER_AL t4;
```

Technical Explanation: Figure 18

- **Description:** Complete defense-in-depth security model spanning perimeter access, per-case cryptographic envelope encryption, air-gapped GPU inference, and append-only ledger immutability.
- **Input:** User credentials, cryptographic keys, storage operations.
- **Processing:** Authentication, envelope encryption, air-gapped model execution, and tamper-evident audit logging.
- **Output:** Protected evidentiary data, secured inference channels, and verifiable logs.
- **Data Movement:** Network traffic restricted to authenticated mTLS tunnels; zero egress from GPU nodes.
- **Technology:** Keycloak, WireGuard, HashiCorp Vault / HSM, LUKS, Immudb.

FIGURE 19 — Implementation Dependency Graph

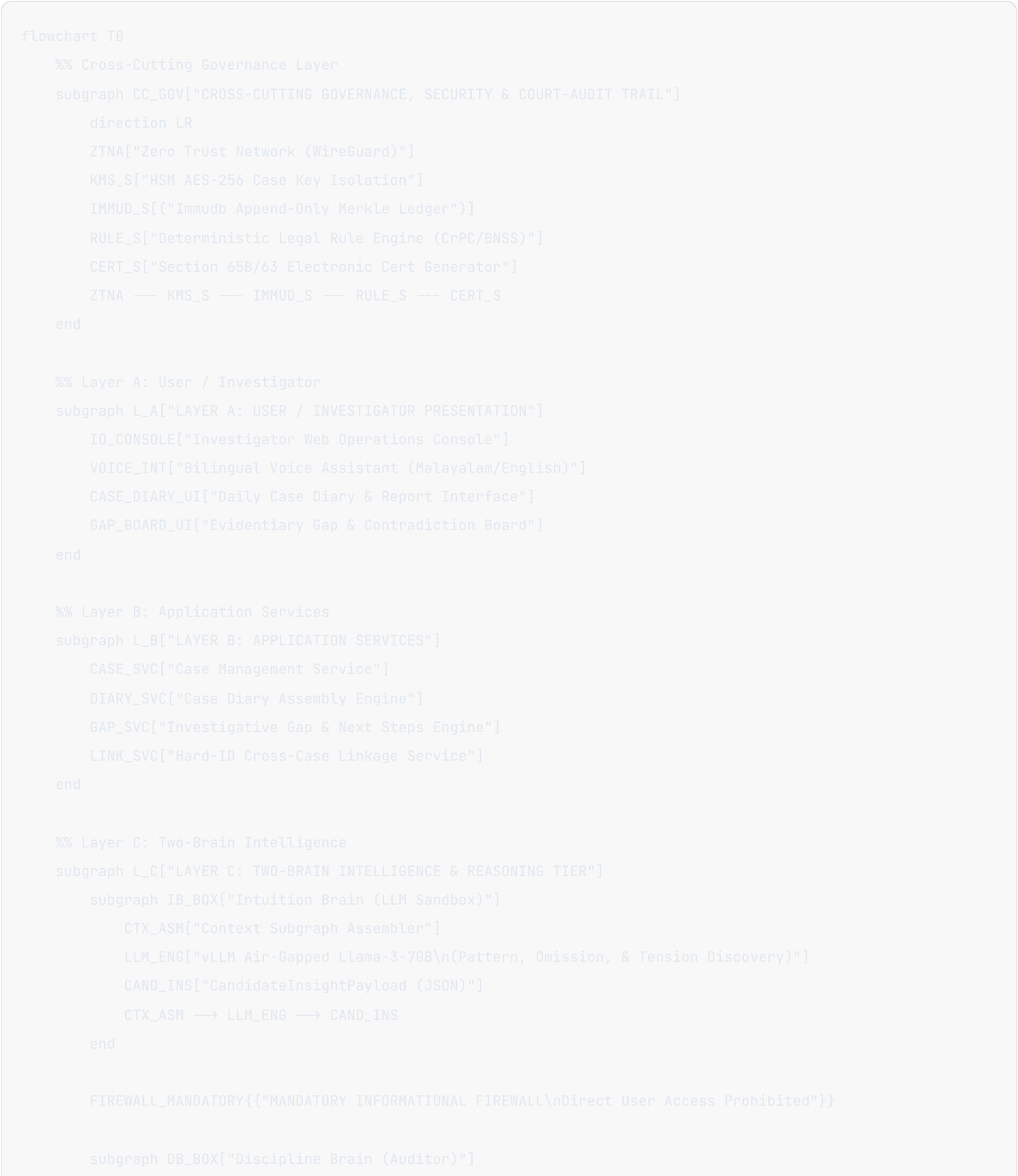


Technical Explanation: Figure 19

- **Description:** Engineering dependency graph mapping the 22 implementation phases across sequential dependencies and parallel cross-cutting infrastructure tracks.
- **Input:** Engineering specifications and component prerequisites.
- **Processing:** Sequential modular construction with strict verification gates at each transition.

- **Output:** Production deployment readiness roadmap.
- **Data Movement:** Upstream phase deliverables serve as architectural inputs for downstream components.
- **Technology:** Jira, GitLab CI, Docker, Kubernetes.

FIGURE 20 — FINAL MASTER SYSTEM ARCHITECTURE



```

CLAIM_DEC["Atomic Claim Decomposer"]
SPAN_VER["Document Span Grounding & Entailment Engine"]
GATE_EVAL["Grounding Ratio = 1.0\n& Forensically Consistent?"]
CLAIM_DEC → SPAN_VER → GATE_EVAL

end

CAND_INS → FIREWALL_MANDATORY → CLAIM_DEC

end

%% Layer D: Case Memory
subgraph L_D["LAYER D: STRUCTURED CASE MEMORY"]
    POSTGRES_DB["PostgreSQL 16 Case DB\n(Entities, Events, Milestones)"]
    NEO4J_KG["Neo4j Knowledge Graph\n(Entity-Event Property Graph)"]
    TIMELINE_ENG["Chronological Timeline & Spatial Interval Engine"]
    QDRANT_VEC["Qdrant Vector Store\n(Paragraph Embeddings)"]
    CLAIM_MAP_DB["Claim-Evidence Mapping Store\n(Grounding Lineage)"]
end

%% Layer E: Evidence Processing
subgraph L_E["LAYER E: DATA PROCESSING & EXTRACTION"]
    OCR_PIPE["Multi-Modal OCR (Tesseract / PaddleOCR)"]
    ASR_PIPE["Bilingual ASR (Malayalam-English Whisper)"]
    NLP_PIPE["Legal NER & ISO 8601 Temporal Parser"]
    ID_EXTRACT["Hard-Identifier Parser (IMEI, Phone, Plate, Acc)"]
end

%% Layer F: Evidence Ingestion
subgraph L_F["LAYER F: MULTI-MODAL EVIDENCE INGESTION"]
    RAW_SOURCES["Evidence Influx: FIR, Statements, Post-Mortem, FSL Lab Reports, CDR, CCTV, Bank Statements"]
    INGEST_CTRL["Ingestion Service & Anti-Virus (ClamAV)"]
    HASHER_DUAL["Dual Hashing Engine (SHA-256 & SHA-3-512)"]
    MINIO_WORM["MinIO WORM Object Vault\n(S3 Compliance Object Lock)"]
end

%% Connections Between Tiers
RAW_SOURCES → INGEST_CTRL → HASHER_DUAL
HASHER_DUAL → |Raw Stream| MINIO_WORM
HASHER_DUAL → |Checksum & Metadata| IMMUD_S

MINIO_WORM → OCR_PIPE & ASR_PIPE
OCR_PIPE & ASR_PIPE → NLP_PIPE & ID_EXTRACT
NLP_PIPE & ID_EXTRACT → POSTGRES_DB & NEO4J_KG & QDRANT_VEC
POSTGRES_DB & NEO4J_KG → TIMELINE_ENG

TIMELINE_ENG & NEO4J_KG → CTX_ASM
MINIO_WORM & POSTGRES_DB -.→ |Grounding Query| SPAN_VER

GATE_EVAL → |Verified Facts / Gaps / Contradictions| CLAIM_MAP_DB
CLAIM_MAP_DB → GAP_SVC & DIARY_SVC & LINK_SVC

```

```
ID_EXTRACT → LINK_SVC

L_B → L_A

%% Closed-loop feedback
IO_CONSOLE →|Human Decisions & New Evidence Collection| RAW_SOURCES
IO_CONSOLE -.→|Log Officer Override| IMMUD_S
IMMUD_S → CERT_S

%% Cross-cutting linkages
CC_GOV -. L_A
CC_GOV -. L_B
CC_GOV -. L_C
CC_GOV -. L_D
CC_GOV -. L_E
CC_GOV -. L_F

classDef gov fill:#09090b,stroke:#27272a,stroke-width:2px,color:#a1a1aa;
classDef la fill:#18181b,stroke:#38bdf8,stroke-width:2px,color:#f0f9ff;
classDef lb fill:#0f172a,stroke:#38bdf8,stroke-width:1.5px,color:#fff;
classDef lc_brain fill:#311042,stroke:#a21caf,stroke-width:2px,color:#fae8ff;
classDef lc_gate fill:#064e3b,stroke:#059669,stroke-width:2px,color:#ecfdf5;
classDef ld fill:#1e1b4b,stroke:#4338ca,stroke-width:2px,color:#e0e7ff;
classDef le fill:#1e293b,stroke:#64748b,stroke-width:1.5px,color:#fff;
classDef lf fill:#022c22,stroke:#059669,stroke-width:2px,color:#fff;

class CC_GOV,ZTNA,KMS_S,IMMUD_S,RULE_S,CERT_S gov;
class L_A,IO_CONSOLE,VOICE_INT,CASE_DIARY_UI,GAP_BOARD_UI la;
class L_B,CASE_SVC,DIARY_SVC,GAP_SVC,LINK_SVC lb;
class IB_BOX,CTX_ASH,LLM_ENG,CAND_INS lc_brain;
class DB_BOX,CLAIM_DEC,SPAN_VER,GATE_EVAL,FIREWALL_MANDATORY lc_gate;
class L_D,POSTGRES_DB,NEO4J_KG,TIMELINE_ENG,QDRANT_VEC,CLAIM_MAP_DB ld;
class L_E,OCR_PIPE,ASR_PIPE,NLP_PIPE,ID_EXTRACT le;
class L_F,RAW_SOURCES,INGEST_CTRL,HASHER_DUAL,MINIO_WORM lf;
```

Technical Explanation: Figure 20 (Master System Architecture)

- **Description:** The master architecture diagram synthesizing the entire Sakshi engineering ecosystem. Unifies Layers A through F with cross-cutting governance, showing how data transforms from raw evidence to court-auditable, verified intelligence.
- **Input:** All statutory police case files, digital artifacts, voice notes, and laboratory findings.
- **Processing:** Complete closed-loop workflow: Ingestion → Dual-Hashing → WORM Storage → Multi-Modal OCR/ASR → NLP/IE → Structured Memory & Graph → Two-Brain Reasoning & Evidence Gating → Application Services → Investigator Dashboard → Closed-Loop Field Re-Ingestion.
- **Output:** Fully auditable, evidence-backed death investigation environment operating under the Cardinal Anti-Suspect Principle.

- **Data Movement:** Strict unidirectional flow across security boundaries; cross-cutting Merkle logging records all state transitions; real-time feedback loops support continuous case refinement.
 - **Technology:** MinIO (WORM), PostgreSQL 16, Neo4j Enterprise, Qdrant, vLLM (Llama-3-70B), Faster-Whisper, Immudb, Keycloak, Next.js, FastAPI, Docker, Kubernetes.
-

Conclusion & Architectural Sign-Off

This document constitutes the complete, definitive system architecture, data flow specification, and engineering implementation roadmap for **Sakshi**. By embedding the **Cardinal Anti-Suspect Principle** into the kernel, isolating generative synthesis within the **Two-Brain Asymmetric Sandbox**, enforcing strict **1.0 Grounding Ratio Evidence Gating**, restricting cross-case linkage to **machine-verifiable hard identifiers**, and maintaining an **append-only Merkle audit trail**, Sakshi provides an objective, court-admissible, and technologically rigorous foundation for modern digital death investigation.